PA2 Unsupervised Learning Report (Clustering)

Part 1: K-Means Clustering

Explanation of functions

Load_data()
This function basically reads the `data2D.npy` file and split the data into both train and validation data with the ration of 75:25.

train_kmean_torch(train_data, k = 5, lr=0.1, epoch=150)
This function first initialize the k cluster means m as a trainable tensor and then converts training data to float PyTorch tensor. It also defines an Adam optimizer with learning rate `lr`. It then does training epochs to compute the squared distance from every data point to its own cluster mean and takes the minimum for each point across k clusters. It then averages them and calculates training loss `L_train` and runs backpropagation and Adam optimizer to update m. Finally, it converts final loss and means to NumPy and returns final training objective value and cluster means.

evaluate(test_data, m)
This function creates a distance matrix with one column per cluster mean and fills each column with squared distances from all test points to that mean. It then takes the minimum distance per point (nearest cluster assignment) and returns the average of those minimum distances as a scalar score.

get_association(test_data, m)
This function's purpose is to assign each data point to its nearest cluster center.

test_pytorch(train_data, test_data, k=5)
This function calls `train_kmean_torch()` to learn k cluster means from the training data, then uses `get_association()` to assign each test point to its nearest cluster. It appends the cluster assignments to the test data for visualization, computes the test set objective via `evaluate()`, prints the resulting test score, and plots each cluster in a different color by iterating over cluster assignments and scattering points with corresponding colors.

test_sckitlearn(train_data, test_data, k=5)
This function fits scikit-learn's KMeans on the training data with lloyd algorithm and maximum 5000 iterations, then predicts cluster labels for each test point. It appends the predicted labels to the test data, computes the test set objective using the learned cluster centers via `evaluate()`, prints the resulting test score, and visualizes the clustering by plotting each cluster in a different color.

Experiment Results

These are the plots for each $k = 1, 2, 3, 4, 5$ comparing our PyTorch implementation and the scikit-learn implementation. For each k, the left panel is PyTorch and the right panel is scikit-learn.

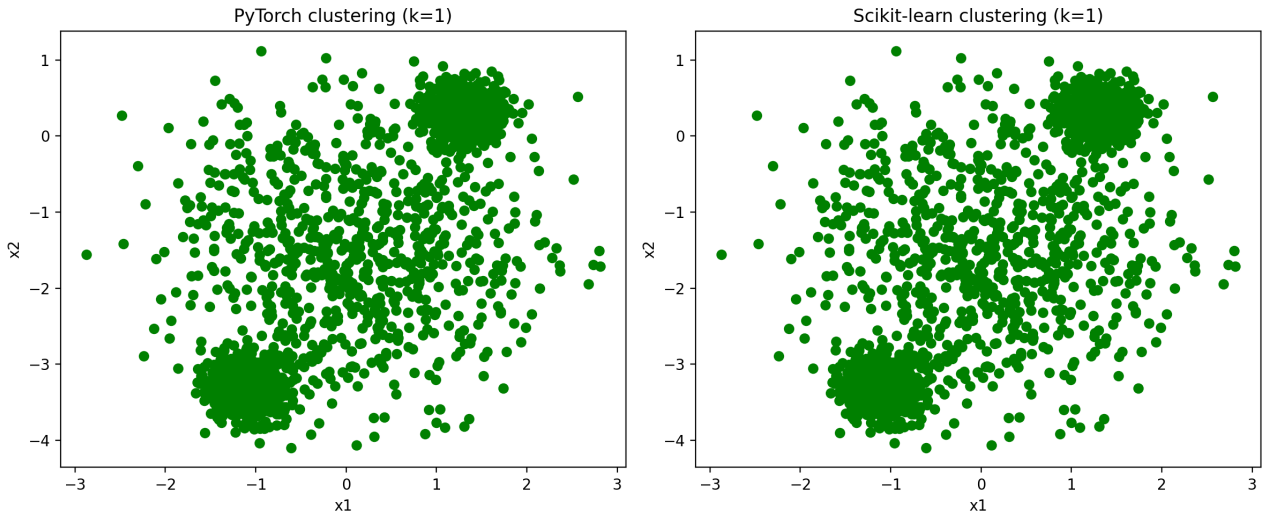

Figure 1: $k = 1$ (left is PyTorch, right is scikit-learn). For $k = 1$, both implementations produce essentially the same result, where all data points are assigned to one cluster center near the global center of the dataset. This is expected because there is only one cluster. The objective values are also almost identical (PyTorch: 3.886296, scikit-learn: 3.886250).

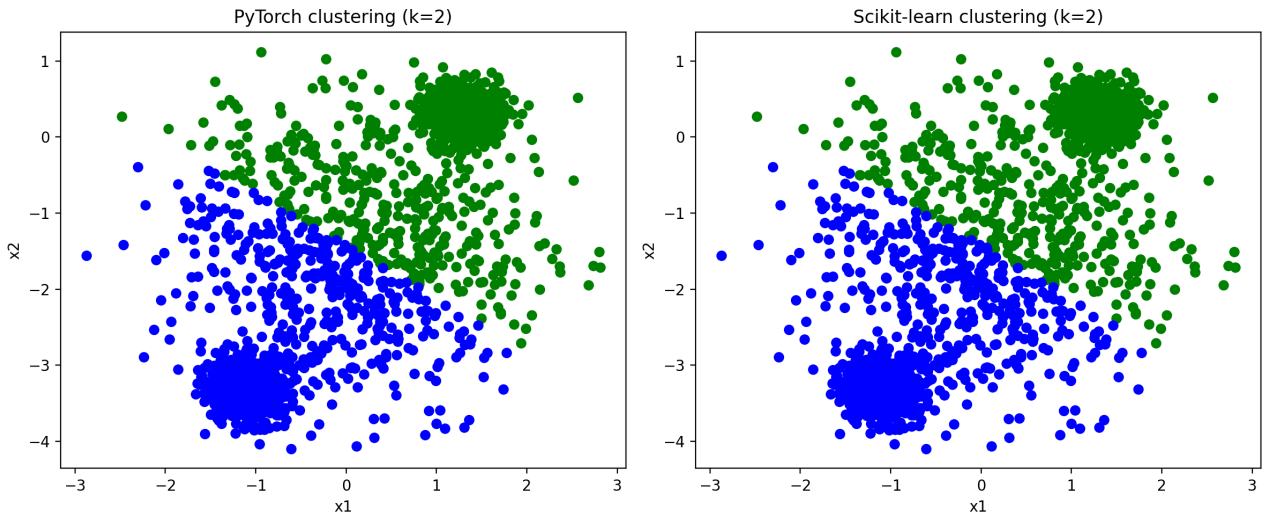

Figure 2: $k = 2$ (left is PyTorch, right is scikit-learn). For $k = 2$, both methods split the data into one compact bottom-left cluster and one larger cluster covering most remaining points. The visual partition is very similar, and the scores are nearly equal (PyTorch: 0.874930, scikit-learn: 0.874963), indicating stable behavior for this setting.

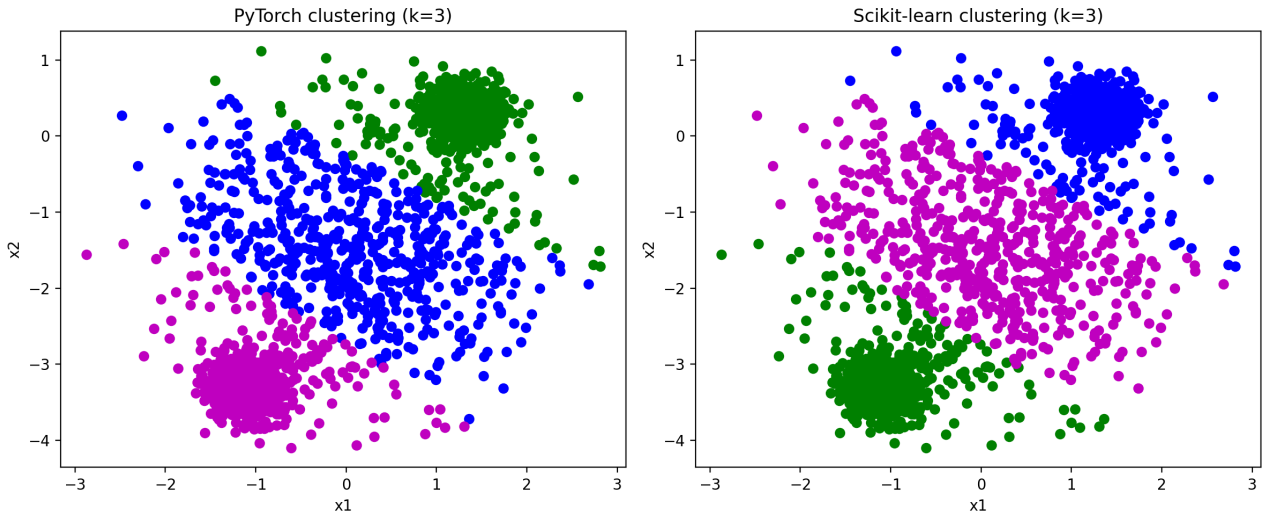

Figure 3: $k = 3$ (left is PyTorch, right is scikit-learn). For $k = 3$, both methods separate the two compact groups (bottom-left and top-right) and assign the remaining points to a broader third cluster. The clustering structures are close, with only small boundary differences. The objective values remain close (PyTorch: 0.488535, scikit-learn: 0.488278).

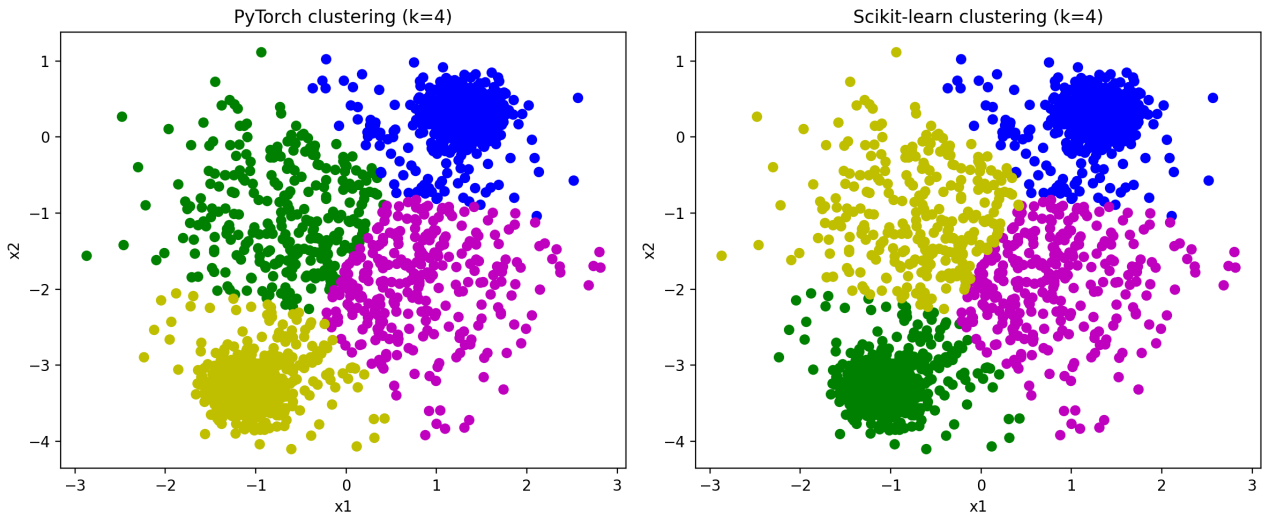

Figure 4: $k = 4$ (left is PyTorch, right is scikit-learn). For $k = 4$, each implementation further subdivides the large central region into multiple parts while maintaining compact clusters near dense regions. Both produce comparable visual quality and similar loss values (PyTorch: 0.315137, scikit-learn: 0.314904).

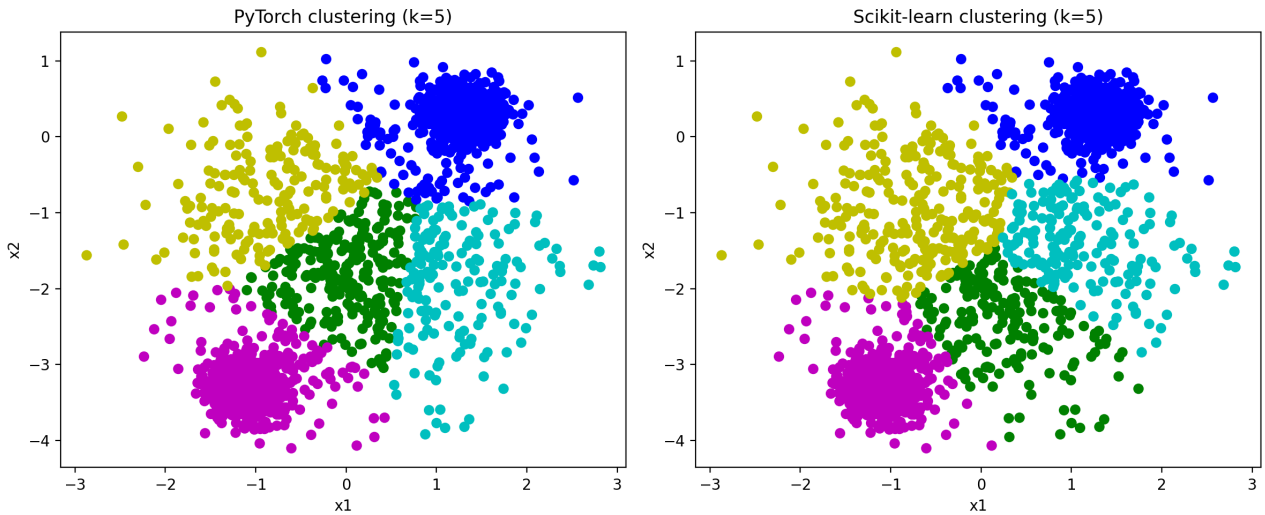

Figure 5: $k = 5$ (left is PyTorch, right is scikit-learn). For $k = 5$, both methods continue to refine partitions, reducing the average nearest-center squared distance. Differences are minor and mainly due to initialization and local minima. The scores remain close (PyTorch: 0.265712, scikit-learn: 0.264583).

Overall Comparison and Comments

Across all k, PyTorch and scikit-learn show very similar clustering behavior and objective values. As expected, the objective decreases as k increases because more centers allow better fit to the data:

- $k = 1$: around 3.886
- $k = 2$: around 0.875
- $k = 3$: around 0.488
- $k = 4$: around 0.315
- $k = 5$: around 0.265

This confirms that our PyTorch implementation is correct and consistent with scikit-learn on this dataset.

Part 2: MoG

Explanation of functions

distanceFunc(X, MU)
`X` is a PyTorch tensor with a shape (N, D) that contains the data points, where N is the number of points and D=2.
`MU` is also a PyTorch tensor with a shape (K, D) that contains the cluster means, where K is the number of clusters.
The function calculates the squared Euclidean distance between every data point and every cluster mean, which covers all possible pairings. It does this by using broadcasting by reshaping the tensors so PyTorch can subtract them all at once, instead of looping through each pair manually.
The output is an (N, K) tensor where the entry [n, k] is the squared distance between point n and cluster center k.

log_GaussPDF(X, mu, sigma)
`X` and `mu` is the same as what is mentioned above.
`sigma` is a PyTorch tensor with a shape (K, 1) that contains the standard deviation for each cluster.
The purpose of the function is to compute how likely each data point is under each Gaussian cluster. A point close to a cluster mean with a small spread gets a high probability while a point far away or in a wide cluster gets a lower probability. It is calculating the log probability density.
The output is basically an (N, K) tensor where the entry [n, k] is the log probability of point n belonging to cluster k.

log_posterior(log_PDF, log_pi)
`log_PDF` is a PyTorch tensor with a shape (N, K) that is what is returned by `log_GaussPDF`.
`log_pi` is a PyTorch tensor with the log of the mixing coefficients, with the shape (K,).
The purpose of the function is to calculate the unnormalized log joint, which is how likely each data point belongs to each cluster, and the log marginal, which is the overall likelihood of each data point across all clusters, by using `logsumexp`.
The output is log joint with the shape (N, K) and log marginal with shape (N,).

Experiment Results

These are the plots for each $k = 1, 2, 3, 4, 5$ for test GMM(k, init_kmeans) running them multiple times, setting init_kmeans to both true and false.

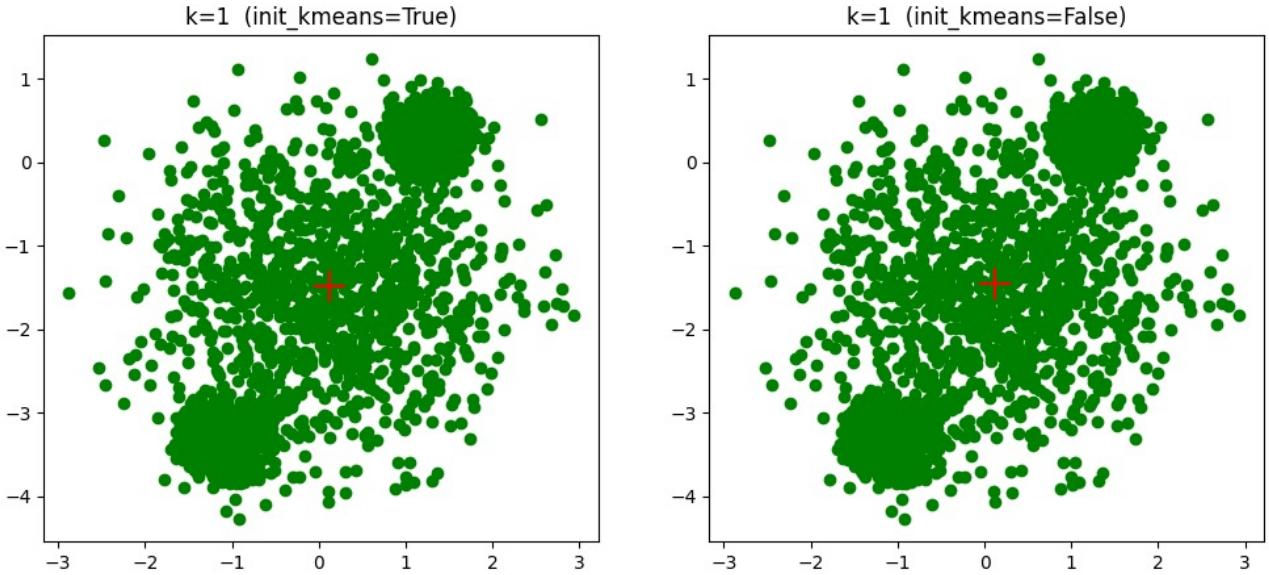

Figure 6: $k = 1$ (left is K-means initialization, right is random initialization). For $k = 1$, both initializations produced basically the same result, where all the data points are assigned to a single Gaussian with the mean is put near the center of the data. There is no effect between the two initializations since there is only 1 cluster.

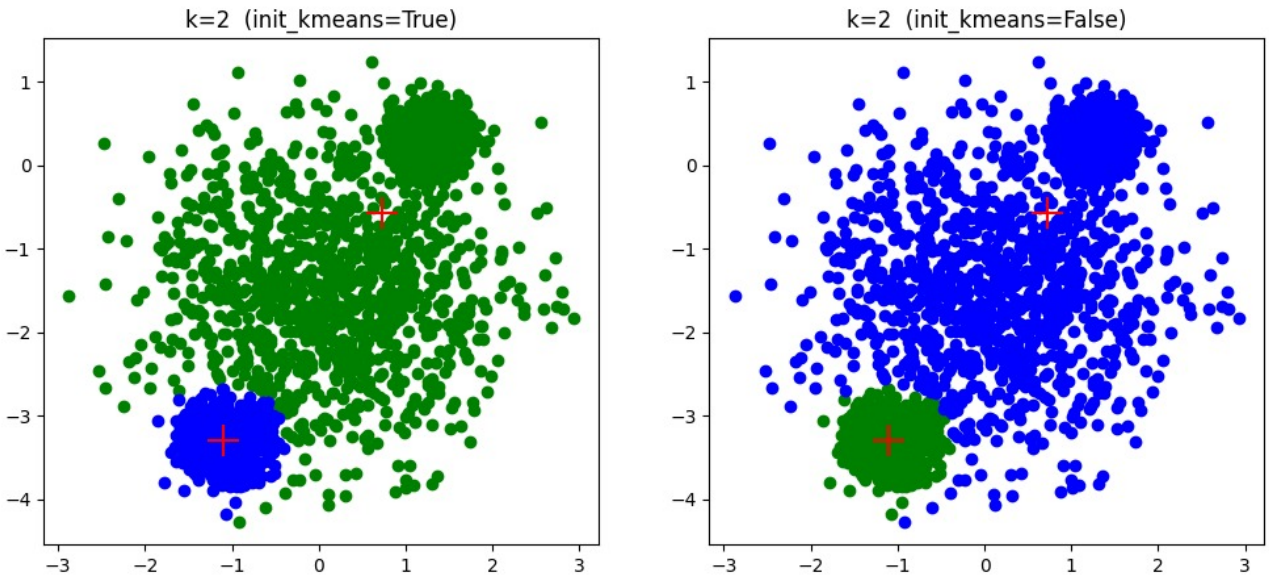

Figure 7: $k = 2$ (left is K-means initialization, right is random initialization). For $k = 2$ the split for both initializations are similar as well, there is one compact cluster at the bottom left and one bigger cluster covering the rest of the data. The results are consistent for both initializations which suggests that $k = 2$ is a stable solution for the dataset.

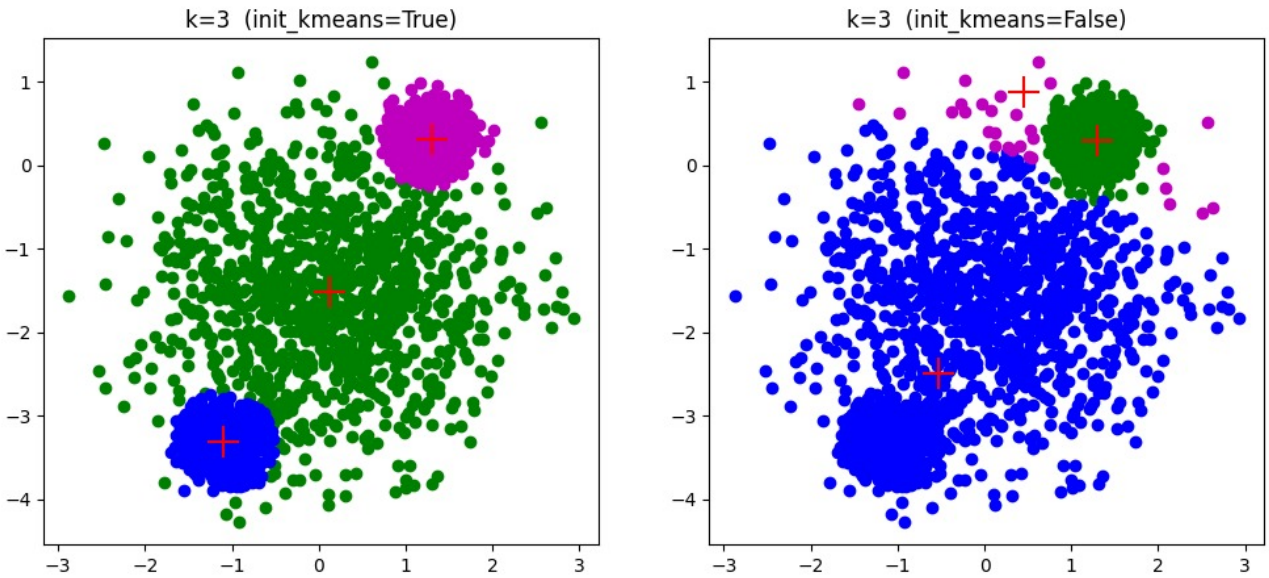

Figure 8: $k = 3$ (left is K-means initialization, right is random initialization). For $k = 3$ for the K-means initialization, there are three clusters, with a compact cluster in the bottom left, another compact cluster at the top right and a large cluster covering the rest of the data, and looks quite reasonable. With random initialization, the only similar cluster is the compact one in the top right, while another cluster dominates almost the entirety of the remaining data, with the third cluster being a small number of scattered points at the top. This shows some weird clustering behaviour.

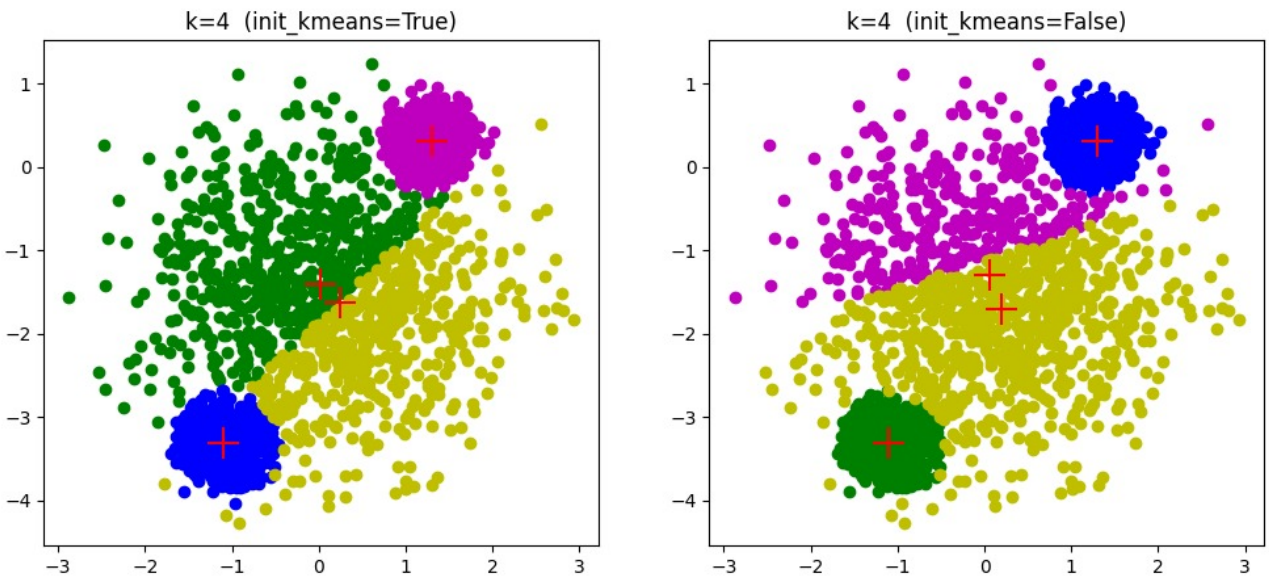

Figure 9: $k = 4$ (left is K-means initialization, right is random initialization). For $k = 4$ with the K-means initialization, it is similar to $k = 3$ but splitting the large cluster (containing the remaining data not in the two compact clusters) into two parts, doing so cleanly and quite evenly. For random initialization, it is also similar to what K-means initialization for $k = 4$ produced except that the split of the large cluster was done so less evenly, with the magenta cluster being smaller than the other.

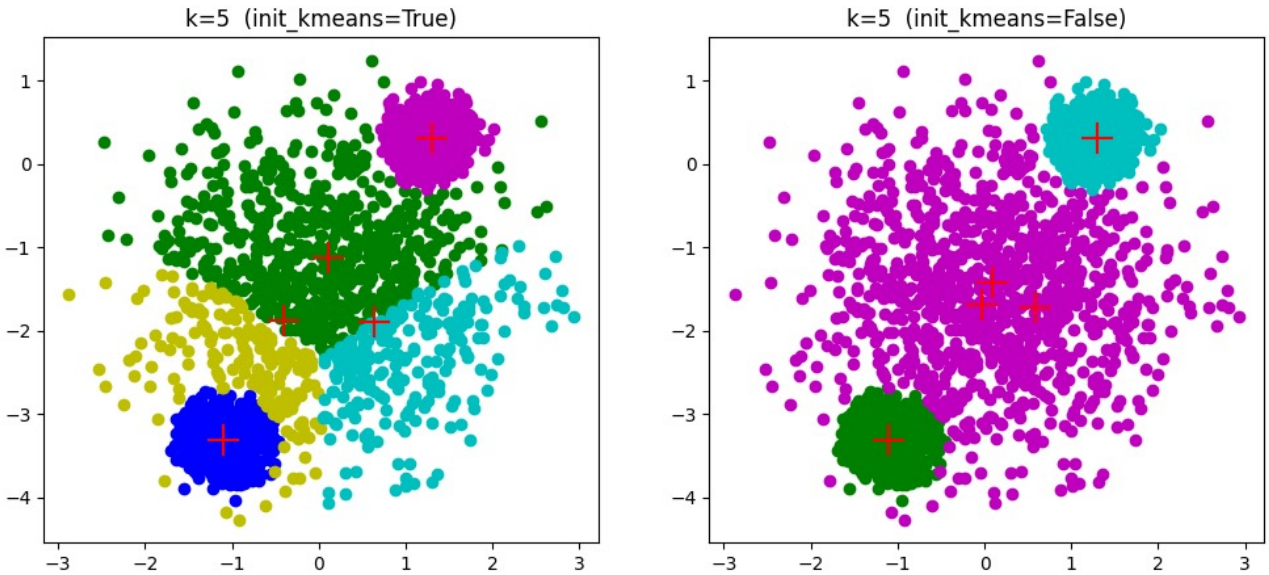

Figure 10: $k = 5$ (left is K-means initialization, right is random initialization). For $k = 5$, with K-means initialization, It is similar to $k = 4$ but with the larger cluster split into 3 parts cleanly and quite evenly, with centers that are separated well. With random initialization, other than the two compact clusters in the top right and bottom left, one magenta cluster dominates almost the entire dataset, with 3 centers close to each other in the middle of the large cluster, wasting 2 of the clusters.

Conclusion
Hence, overall, we see here that K-means produces clusters that are more balanced and reasonable while random initialization makes it prone for clusters to collapse into a single dominant cluster, which is the ”weird clusters” we see when we set `init_kmeans` to false.